

Shoku — Regression Test Plan

Product: Shoku — multi-tenant, white-label café ordering + POS SaaS (Next.js 14 App Router, Prisma/SQLite, NextAuth JWT, Razorpay). Live at getshoku.com.

Purpose: A repeatable suite to run before every release so shipped changes don't break existing behaviour. Prioritised so a time-boxed run still covers the money- and data-critical paths.

1. Scope & objectives

In scope: customer storefront, café admin console, POS module, super-admin console, all `app/api/*` routes, auth/tenancy, payments, loyalty, WhatsApp, cron, print routes, research/guide pages, deploy scripts.

Out of scope (separate checklists): ad-asset pipeline (`shoku-ads/`), marketing-site copy, infra provisioning (DNS/nginx/cert), the pitch-deck contents.

Objectives

1. No regression on **money math** (order totals, tax/GST, discounts, invoices).
2. No regression on **tenant isolation** — one café can never read/write another's data.
3. No regression on **auth boundaries** — role gates and cross-tenant login blocks hold.
4. Core user journeys (order → pay, POS bill, provision café) complete end-to-end.
5. The **9 audit fixes** stay fixed (see §9 — these are mandatory in every run).

Priorities

- **P0** — money, data integrity, tenant isolation, auth, checkout/POS happy path. Block release on any failure.
- **P1** — feature correctness users would notice (analytics, loyalty, import, marketing).
- **P2** — cosmetic, edge robustness, nice-to-have.

2. Environments & entry criteria

Env	Use	DB
Local dev	full suite, destructive tests	<code>file:./dev.db</code> , freshly seeded
Staging (VPS mirror)	smoke + release checklist	copy of prod schema, seeded demo data
Production (getshoku.com)	read-only smoke only (health, load a storefront)	live — never run write/destructive tests

Entry criteria: `npm run setup` succeeds, `npm run build` passes, `npx vitest run` green (currently **36/36**), seed provides `super@shoku.app`, `demo@shoku.app`, `owner@bluetokai.app`, customers (all password password).

Exit criteria: all P0 pass, no open P1 regressions, `/api/health 200` on target env.

3. Test approach (pyramid for this codebase)

E2E (Playwright, headless)
Integration (route handlers)
Unit (Vitest, pure libs)

few — full journeys per role
medium — API + Prisma + auth, seeded DB
many — money/loyalty/pos/pricing/sanitize

- **Unit** — pure functions: `lib/posMath` (`gstSplit`, `invoice format`), `lib/menu.resolveUnitPrice`, `lib/loyalty`, `lib/ai`, `lib/payments` (`signature`), `lib/sanitize`, `lib/rateLimit`, `lib/segments`. Fast, run on every commit.
- **Integration** — call route handlers against a seeded test DB with a mocked session; assert status + DB state. Best ROI for `tenancy/auth/money`.
- **E2E** — Playwright drives the three roles through critical journeys on a dev server (pattern already used for verification this project).
- **Manual exploratory** — 30-min pass on new UI each release.

4. Regression suites by area

Each row: **area** — **what to verify** — **type** — **priority**. Test-case IDs (e.g. ORD-1) are referenced in §9/§10.

4.1 Auth & session (P0)

- **AUTH-1** Login with correct creds on the right host → session with correct `role` + `tenantId`. (*integration/E2E*)
- **AUTH-2** Café user cannot log into a *different* café's subdomain (host-scoped `authorize`). (*integration*) — **tenant isolation**
- **AUTH-3** Superadmin blocked on a café subdomain; only resolves at apex. (*integration*)
- **AUTH-4** Wrong password rate-limited after N attempts per IP+email. (*integration*) — see `rateLimit.test`
- **AUTH-5** `requireAdmin` / `requireSuperadmin` reject wrong-role and no-session with 401/403. (*integration*)
- **AUTH-6** Invite accept (`/api/invite/accept`) + `/set-password` sets a usable password once; link not reusable. (*E2E*)
- **AUTH-7** Register creates a customer scoped to the current tenant only. (*integration*)
- **AUTH-8** `JWT/x-tenant-slug` cannot be spoofed by a client header — middleware overwrites it from the validated host. (*integration*) — **security**

4.2 Multi-tenancy isolation (P0) — run against ≥2 seeded cafés

For **every** authenticated API route, assert a café-A session cannot read or mutate café-B rows.

- **TEN-1** GET `/api/menu`, `/api/admin/orders`, `/api/admin/customers`, `/api/admin/analytics`, `/api/admin/stats`, `/api/admin/feedback` return only current-tenant data. (*integration*)
- **TEN-2** PATCH/DELETE on `items/[id]`, `orders/[id]`, `discounts/[id]`, `loyalty/rewards/[id]`, `tables/[id]`, `banners/[id]` with a **foreign id** → 404/no-op (never edits the other tenant). (*integration*)
- **TEN-3** Print routes `/print/invoice/[id]`, `/print/kot/[id]` refuse a foreign order id. (*integration*)
- **TEN-4** `createOrder` rejects line item ids that belong to another tenant (filtered by `tenantId`). (*integration*)
- **TEN-5** Storefront on a suspended tenant shows "unavailable" and blocks ordering. (*E2E*)

4.3 Customer storefront & ordering (P0)

- **MENU-1** Menu loads, categories/filters/search work, only `live` items show. (*E2E*)
- **ITEM-1** Item page: size + milk selection updates displayed price per catalog. (*E2E*)
- **CART-1** Add/increment/decrement/remove; cart persists across reload (`localStorage`); count/subtotal correct. (*E2E/unit*)
- **CART-2** Corrupt `localStorage` (`shoku.cart` = bad JSON) → app still loads, empty cart, no crash. (*E2E*) — robustness
- **ORD-1** Checkout happy path (mock-paid): totals on screen == server order; status `preparing`; points awarded. (*E2E+integration*) — **money**

- **ORD-2** Empty cart → checkout blocked ("Cart is empty"). (*integration*)
- **ORD-3** Item deleted mid-checkout → order created without it (or clean error), never a 500. (*integration*)
- **ORD-4** Dine-in via table QR records `tableLabel`; fulfilment `dinein`. (*E2E*)
- **ORD-5** Order status advance (preparing→ready→completed→cancelled) via admin; customer/account view reflects it. (*E2E*)

4.4 Money math & discounts (P0) — mostly unit + integration

- **PAY-1** `total = max(0, subtotal + tax - reward - discount - loyaltyDiscount); tax = tenant.gstRate%`. (*unit/integration*)
- **PAY-2** Promo code: valid % applies; invalid/inactive/foreign-tenant code ignored; case/space-insensitive. (*integration*)
- **PAY-3** Displayed checkout total == charged total for `gstRate ≠ 5` (e.g. 18%). (*E2E*) — **audit fix, §9-3**
- **PAY-4** Stacked 90% promo + full loyalty reward → total floors at 0, never negative; points still consistent. (*integration*) — **audit fix, §9-2**
- **PAY-5** Line price always from catalog; client-sent unit ignored. (*unit resolveUnitPrice + integration*) — **audit fix, §9-1**

4.5 Payments — Razorpay (P0)

- **RZP-1** `verifyPaymentSignature` accepts a correctly-signed payload, rejects tampered ones (timing-safe). (*unit — payments.test*)
- **RZP-2** `/payments/razorpay/order` creates a pending order with server-computed amount. (*integration*)
- **RZP-3** `/verify` marks paid + awards points **once**; replay is idempotent. (*integration*)
- **RZP-4** `/webhook` rejects bad signature (400) before parsing; `payment.captured` settles; `payment.failed` marks failed **and refunds reserved loyalty points once**. (*integration*) — **audit-related, §9-4 partner**
- **RZP-5** Keys unset → graceful mock-paid flow still works (no crash). (*integration*)

4.6 Loyalty (P0/P1)

- **LOY-1** Earn rate: `points = floor(subtotal * earnRate/100)` on paid order. (*unit/integration*)
- **LOY-2** Redeem reward: points reserved atomically at order creation; balance can't go negative under concurrent double-submit. (*integration, concurrency*) — **audit fix, §9-4**
- **LOY-3** Failed/again-failed payment refunds redeemed points exactly once. (*integration*)
- **LOY-4** Tiers computed from config; fallback to `DEFAULT_TIERS` when unset/garbled. (*unit — loyalty.test*)
- **LOY-5** `freeItem` reward adds a ■0 line; discount reward caps at subtotal. (*integration*)

4.7 POS module (P0) — requires `posEnabled`

- **POS-1** `requirePos` gate: POS routes/pages 403 when add-on off; work when on. (*integration*)
- **POS-2** Ring up a sale: `/pos/orders` creates order `source=pos`, assigns sequential `invoiceNo`, splits `cgst/sgst` summing to `tax`. (*integration + unit posMath*) — **money**
- **POS-3** Phone-matched customer earns loyalty; anonymous walk-in uses `skipLoyalty` (no points, no WhatsApp). (*integration*)
- **POS-4** Two concurrent bills get **distinct** invoice numbers (atomic `invoiceSeq` increment). (*integration, concurrency*)
- **POS-5** Invoice number **resets on year rollover**, legacy `invoiceYear=0` adopts current year without reissuing. (*unit/integration*) — **audit fix, §9-7**
- **POS-6** Day-end (Z) report: totals, payment split, POS/online counts, top items correct; date **label = local calendar day** (not UTC-shifted). (*unit/integration*) — **audit fix, §9-6**

- **POS-7** `/print/invoice/[id]` shows GSTIN + CGST/SGST + totals; `/print/kot/[id]` shows items+qty, no prices; both outside admin chrome. (E2E/visual)
- **POS-8** `/pos/settings` PUT (owner) updates `gstin/gstRate(0-28)/prefix/kotAutoPrint`; `posEnabled` only via superadmin. (integration)

4.8 Menu management & CSV import (P1)

- **IMP-1** Template download + upload → preview → import; created/updated counts correct; images auto-assigned. (E2E)
- **IMP-2** Same item name in **two categories** creates two items (category-disambiguated id), not an overwrite. (integration) — **audit fix, §9-5**
- **IMP-3** Bad rows (missing name/price, price ≤ 0, non-numeric) reported in errors, valid rows still imported. (integration)
- **IMP-4** `foodImages.imageFor` keyword match order: "chai latte" → chai photo, not latte. (unit)
- **IMP-5** AI image path: with key → `generateMenuImage` used, falls back to stock on failure; without key → stock, `aiAvailable=false`. (integration, mock fetch)
- **IMP-6** Row cap (≤200) enforced; oversized/garbage CSV rejected cleanly. (integration)
- **IMP-7** Manual add/edit/delete item; delete blocked (409) when referenced by past orders. (integration)

4.9 Analytics & locations (P1)

- **AN-1** `/api/admin/analytics?days=7|30|90`: KPIs, revenue-by-day, orders-by-hour, POS-vs-online + method split, top items — match seeded data. (integration)
- **AN-2** Revenue-by-location populates when tenant has `locations` and orders carry `locationLabel`; empty-state otherwise. (integration) — regression: this was blank on a stale Prisma client; verify on fresh boot.
- **AN-3** Growth = this-half vs prior-half; null when no prior data. (unit)
- **LOC-1** Settings: add/remove locations (JSON validated, capped); storefront shows picker only when ≥1. (E2E)
- **LOC-2** Removed location **resyncs** — no longer stuck in cart or recorded on new orders. (E2E) — **audit fix, §9-8**
- **LOC-3** Chosen location persisted to order `locationLabel` and reflected in AN-2. (integration)

4.10 Marketing / WhatsApp (P1)

- **WA-1** Demo mode (no provider) logs messages, never sends; UI shows demo banner. (integration)
- **WA-2** Campaign send builds the segment audience (cap 500), personalises, records messages + audit. (integration)
- **WA-3** `campaigns/suggest` returns AI copy when keyed, heuristic otherwise. (integration, mock)
- **WA-4** `wa/settings` GET masks secrets; PATCH updates provider/creds. (integration)
- **WA-5** `cron/nudges` gated (CRON_SECRET or superadmin), de-dupes recent nudges, respects `waNudges`. (integration)
- **WA-6** Segment counts (`campaigns/segments`) match `lib/segments` logic. (unit/integration)

4.11 Super-admin (P0/P1)

- **SUP-1** Provision café: creates tenant + starter categories/menu + owner (password or invite link); audit logged. (integration/E2E)
- **SUP-2** Reserved/taken/invalid slug rejected; slug regex enforced. (integration)
- **SUP-3** Suspend/activate toggles storefront availability. (E2E)
- **SUP-4 Manage drawer**: plan change, POS add-on toggle, AI key/model/baseUrl set; empty string clears to defaults; `aiKeySet` masked. (integration/E2E)

- **SUP-5** `getTenantLLMConfig` resolves tenant override → plan default → env, per plan. (*unit*)
- **SUP-6** Platform analytics + audit log render with real aggregates. (*E2E*)

4.12 Content pages (P2)

- **DOC-1** `/guide` shows café tab always; **demo playbook** + **developer** tabs only for staff roles. (*E2E*)
- **DOC-2** Markdown renders sanitised (no XSS via `safeMarkdownHtml`). (*unit* — *sanitize.test* + *integration*)
- **DOC-3** `/research` bands + charts render; `/pitch` deck loads; pitch upload (super) version-increments. (*E2E*)
- **DOC-4** `/api/health` returns 200 + status JSON. (*smoke*)

4.13 Cross-cutting security (P0)

- **SEC-1** CSP + security headers present (`next.config`). (*integration*)
- **SEC-2** Markdown/user text sanitised everywhere it's rendered as HTML. (*unit*)
- **SEC-3** Rate limits on login + register; limiter memory pruned >5000 keys. (*unit*)
- **SEC-4** `imageGen` filename can't traverse (slug regex-validated). (*unit/integration*)
- **SEC-5** No secret leakage in API responses (wa creds, `aiApiKey`, password hashes). (*integration*)

5. Concurrency & data-integrity tests (P0, easy to miss)

Run these deliberately — they cover the race classes the audit found:

- **CON-1** Two simultaneous `createOrder` redeeming the same reward → exactly one succeeds in reserving points; balance never negative. (§9-4)
- **CON-2** Two POS terminals billing at once → no duplicate/skipped `invoiceNo`. (§9-7 *partner*)
- **CON-3** `markOrderPaid` called twice (client verify + webhook) → points awarded once. (*RZP-3*)
- **CON-4** `seed-demo-data.js` run while POS is live → uses atomic increment, no invoice collision. (§9-9)

Technique: fire N parallel `Promise.all` requests against the route/handler on a seeded DB; assert final DB state.

6. Non-functional (P1/P2)

- **PERF-1** Menu + analytics endpoints respond < 500ms on seeded data. (*smoke timing*)
- **PERF-2** CSV import of 200 rows completes within `maxDuration` (120s) even with AI images off. (*integration*)
- **BUILD-1** `npm run build` clean, no type/lint breakage. (*CI*)
- **A11Y-1** Storefront + checkout keyboard-navigable, sufficient contrast (brand tokens). (*manual/axe*)
- **COMPAT-1** Storefront usable on mobile viewport (375px) — primary customer surface. (*E2E resize*)

7. Release smoke checklist (run on staging/prod after every deploy)

1. `curl https://<host>/api/health` → 200.
2. Load a café storefront (apex + one subdomain) → menu renders, brand theme applied.
3. Log in as owner → dashboard + analytics load with data.
4. Place one test order (mock) → appears in admin Orders.
5. (POS café) Ring one bill → invoice prints, day-end increments.
6. Super-admin → Cafés list loads with live revenue.

7. Check no console/server errors (`pm2 logs`).
8. Confirm the deploy ran migrations (`prisma db push`) — new columns exist (`invoiceYear`, `locations`, `posEnabled`, `AI` fields).

After a **history-rewrite deploy**, verify collaborators re-synced and the running process restarted (stale Prisma client caused the AN-2 blank-locations bug once).

8. Test data & tooling

- **Seed:** `npm run seed` (2 cafés) + `node scripts/seed-demo-data.js cbtl 30` for realistic analytics/day-end. Second café (Blue Tokai) is essential for all TEN-* isolation cases.
- **Fixtures needed:** a café with `gstRate=18` and `posEnabled`, a café with 3 `locations`, a customer with an exact reward-cost point balance (for LOY-2/CON-1), an item referenced by a past order (IMP-7).
- **Tooling:** Vitest (units, wired), Playwright/Chromium (E2E — already used for headless verification), a thin integration harness that imports route handlers with a stubbed `getServerSession`. `curl/node` scripts for concurrency bursts.
- **Mocking:** `fetch` for LLM/Razorpay/WhatsApp so tests are deterministic and offline.

9. Audit-fix regression cases — mandatory every run

These nine were live defects fixed on `fix/audit-findings`. Each must have a permanent guard so it can never silently regress.

#	Defect	Locked-in test	Area
9-1	Client-controlled line price (buy anything for ■1)	PAY-5 + <code>resolveUnitPrice</code> unit tests (in <code>menu.test</code>)	4.4
9-2	Stacked discounts → negative total	PAY-4	4.4
9-3	Checkout showed 5% while server charged <code>gstRate</code>	PAY-3	4.4
9-4	Loyalty redemption race → negative points	LOY-2 / CON-1	4.6/5
9-5	CSV import same-name-diff-category overwrite	IMP-2	4.8
9-6	Day-end date label off-by-one (UTC vs IST)	POS-6	4.7
9-7	Invoice sequence never reset per year	POS-5	4.7
9-8	Removed location stuck in cart / on orders	LOC-2	4.9
9-9	Demo seeder non-atomic <code>invoiceSeq</code> write	CON-4	5

Bonus regression: **failed-payment point refund** (RZP-4/LOY-3) — introduced alongside the atomic-reservation fix; without it, a failed gateway payment would strand the customer's reserved points.

10. Current coverage & prioritised gaps

Have (Vitest, 36 tests): `ai`, `loyalty`, `menu` (incl. new `resolveUnitPrice`), `payments`, `pos` (`posMath`), `rateLimit`, `sanitize`. These cover PAY-5, PAY-1(partial), RZP-1, LOY-1/4, POS-2(math), IMP-4, SEC-2/3.

Highest-value gaps to automate next (ordered):

1. **Integration harness for tenant isolation (TEN-1..4)** — highest risk, currently only manually verified. (P0)
2. **Order/money integration** (ORD-1/2/3, PAY-2/3/4) against a seeded DB. (P0)

3. Concurrency suite (CON-1..4) — the audit's race fixes have no automated guard yet. *(P0)*

4. POS integration (POS-2/4/5/6 beyond posMath units). *(P0)*

5. CSV import integration (IMP-2/3/5/6). *(P1)*

6. E2E journeys for the three roles (Playwright), reusing the existing headless setup. *(P1)*

7. Analytics/locations integration (AN-1/2, LOC-2/3). *(P1)*

Suggested cadence: units on every commit (pre-push); integration + concurrency in CI on PR; full E2E + smoke on release to staging; read-only smoke on prod post-deploy.

11. Sign-off

A release is regression-clean when: **§9 all pass**, all **P0** in §4/§5 pass, no open **P1**, vitest green, `npm run build` clean, and the §7 smoke checklist passes on staging. Record the run (date, commit SHA, pass/fail per suite, defects filed) in the release ticket.